



Guia de Estudos: Lab de Computação

Distribuída

Assunto 1: Sockets, Concorrência e Estado Compartilhado

1. [Nível: Análise] Em um cenário de *stress test* de um servidor de e-commerce simulado, 50 clientes tentam realizar a operação de COMPRAR simultaneamente. O estoque inicial é de 10 itens. Se o servidor for implementado com a arquitetura *Thread-per-Connection* sem o uso de primitivas de sincronização (como Lock ou Mutex), qual das alternativas a seguir descreve corretamente a anomalia arquitetural que ocorrerá e sua causa?

a) O servidor entrará em *deadlock*, pois múltiplas threads tentarão ler a variável de estoque ao mesmo tempo, bloqueando o descritor de arquivo do Socket.

b) Ocorrerá uma Condição de Corrida (*Race Condition*), permitindo que o estoque fique negativo ou que mais de 10 clientes recebam a mensagem de "Compra realizada", pois a leitura e o decremento do estoque não ocorrerão de forma atômica.

c) O servidor rejeitará as conexões excedentes, pois a porta TCP 12346 não suporta mais de 10 descritores de rede simultâneos.

d) O estoque permanecerá consistente, mas o tempo de resposta aumentará linearmente devido ao enfileiramento dos pacotes Unicast no protocolo de transporte TCP.

2. [Nível: Aplicação] Você está implementando a "Atividade 2: Servidor de Leilão". O requisito exige que, ao receber um lance maior que a variável global MAIOR_LANCE, o servidor atualize o valor e notifique TODOS os clientes conectados (Broadcast). Como essa lógica de Broadcast deve ser aplicada no código do servidor em Python?

a) Fechando as conexões atuais e reabrindo um socket UDP configurado com SO_BROADCAST para enviar a string de notificação.

b) Utilizando a função MPI_Bcast para transferir os dados da thread atual para as demais threads gerenciadas pelo interpretador.

c) Mantendo uma lista global iterável de clientes conectados e, dentro da thread que validou o lance (protegida por Mutex), iterar sobre os sockets dessa lista enviando a mensagem "NOVO LANCE" codificada.

d) Retornando a mensagem apenas no socket do cliente atual e confiando que o roteador de borda duplicará os pacotes para os demais IPs.

3. [Nível: Aplicação] Considere o código de sincronização de estoque via Lock em Python. Você precisa modificar a função `realizar_venda()` para que ela processe a devolução de um item (incremento do estoque). Qual a aplicação correta do Lock para essa nova operação?

a) `def devolver(): global estoque; with lock: estoque += 1; return True.`

b) `def devolver(): global estoque; lock.acquire(); estoque += 1; return True` (sem liberação explícita ou bloco *with*).

c) Não é necessário usar Lock para incremento de variáveis inteiras no Python, pois a adição é uma operação atômica de rede.

d) `def devolver(): global lock; with estoque: lock += 1; return True.`

4. No laboratório de Sockets (Atividade 2), foi proposto um "Desafio Reflexivo": o que acontece se dois usuários enviarem lances idênticos (ex: R\$ 500,00) no exato mesmo milissegundo? Analise o problema do ponto de vista do gerenciamento de estado compartilhado e justifique por que a simples separação em múltiplas threads não resolve a consistência dos dados.

R: Condição corrida acontece quando o resultado final de um processo depende da ordem de execução ou do timing de eventos não controlados. Quando por exemplo, em um ambiente de memória compartilhada, várias threads tentam acessar e modificar uma variável ao mesmo tempo. Se acontece de uma Thread A manda um lance e o sistema vê que o valor > lance atual e o SO interrompe a primeira e outra Thread B faz a mesma coisa, o sistema valida os dois lances sendo que isso não é permitido em um sistema de leilão. Para resolver o problema da condição de corrida, é preciso utilizar o mutex para tratar da parte do código onde os lances são analisados como uma seção crítica. Quando uma thread acessa essa seção crítica, o acesso a variável é trancado bloqueando o acesso dessa variável para outras threads até que a primeira thread termine a operação e "destranque" (unlock) o acesso.

5. Durante o Teste de Estresse do Servidor de Vendas, o cliente de simulação precisa enviar comandos e, ao mesmo tempo, manter o socket aberto por alguns segundos para capturar o Broadcast de esgotamento de estoque. Analise os gargalos de implementar essa escuta de forma síncrona (bloqueante) na thread do cliente e sugira qual abordagem assíncrona resolve o problema de travamento.

R: O padrão do socket TCP é bloquear a execução do restante da thread atual quando o método `recv()` é chamado. Enquanto os dados não são recebidos pelo buffer, a conexão é encerrada ou ocorre algum erro, o programa não recebe o controle de volta pelo SO. Dessa forma, o broadcast da forma síncrona não funciona pois ele fica congelado nessa parte. Duas formas de resolver esse problema são utilizando timeouts de socket ou de um forma mais robusta e separando as responsabilidades em threads

diferentes. No timeout de socket, um limite de leitura é configurado e se nada chegar nesse intervalo o `recv()` abre uma exceção que permite que a thread siga com se tem algum comando para enviar até tentar a operação de leitura novamente. A outra abordagem é criando duas threads separadas uma para leitura dos dados do `recv()` que nunca interrompe a lógica de processamento e roda um `while` até o broadcast chegar e outra para de operação de lógica de entrada de dados e envio de pacotes COMPRAR.

Assunto 2: Fundamentos de MPI e Cluster

1. [Nível: Análise] Ao executar comandos em um único nó (localmente) usando o OpenMPI, um aluno notou que o comando `mpirun -np 4 hostname` falhou ao ser executado nativamente, retornando um erro de falta de *slots*, mas funcionou perfeitamente ao adicionar a flag `--oversubscribe`. Considerando a arquitetura de processadores e o ambiente MPI, qual análise explica esse fenômeno?

- a) O MPI exige que todos os processos possuam uma placa de rede dedicada para operar.
- b) O comando nativo requer nós físicos distintos; a flag `--oversubscribe` ativa o modo Multicast para emular IP remoto.
- c) Por padrão, o OpenMPI mapeia cada processo solicitado (`-np`) a um núcleo de processamento físico/lógico (identificado por comandos como `nproc` ou `lscpu`). Se os processos solicitados ultrapassarem os *cores* disponíveis, o MPI barra a execução, a menos que o *oversubscribe* instrua o mapeamento de múltiplos processos no mesmo núcleo.
- d) A flag indica que a memória cache L3 deve ser sobregravada pelos processos excedentes do contêiner Docker.

2. [Nível: Aplicação] Você provisionou um cluster com 4 nós (1 master, 3 workers) e definiu um arquivo chamado `hosts` contendo os slots disponíveis de cada nó. Qual seria o comando correto para aplicar a execução do programa `date` paralelizado em 8 processos distribuídos, garantindo que o MPI consulte o seu arquivo de mapeamento de cluster?

- a) `mpirun -n 8 date --hosts`
- b) `mpirun --hostfile hosts -np 8 date`
- c) `mpirun -np 8 --oversubscribe date`
- d) `docker compose exec master mpirun -cluster date`

3. [Nível: Aplicação] Para que o processo master consiga orquestrar os processos nos workers remotamente de forma fluida no laboratório de cluster Ubuntu, é necessário eliminar a barreira de autenticação por senha manual e a verificação estrita de *host keys*. Como você aplica essa configuração?

- a) Utilizando o comando `chmod 777` em todos os executáveis binários do MPI.

b) Desabilitando o Firewall do Windows no ambiente host e repassando o comando `mpirun`.

c) Gerando uma chave SSH (`ssh-keygen`), adicionando a chave pública (`id_rsa.pub`) no arquivo `authorized_keys` dos *workers*, e configurando o parâmetro `StrictHostKeyChecking` no no arquivo `config` do SSH.

d) Enviando as senhas como parâmetro dinâmico em plain-text no arquivo Dockerfile.

Questões Discursivas

4. Em um dos exercícios do laboratório, solicitou-se descobrir qual máquina na rede (worker) possui a menor latência (ping mais rápido) em relação à máquina Mestre. Analise por que essa métrica de latência de rede é um fator crítico para o desempenho escalável e o tempo de execução (Speedup) de aplicações em MPI (Memória Distribuída), em contraste com aplicações *multithreading* em Memória Compartilhada.

R: No multithreading, todas as threads estão localizadas no mesmo nó e acessam o mesmo barramento de memória. A comunicação é feita através de ponteiros ou variáveis compartilhadas. A latência aqui é medida em **nanossegundos**, limitada apenas pela velocidade do hardware interno.

Já no MPI, cada processo tem seu próprio espaço de endereçamento isolado. Para que os processos diferentes conheçam o valor um do outro, os dados precisam ser, serializados, encapsulados em pacotes TCP, enviados através da rede NIC, passar por switches e reconstruídos novamente no destino. A latência aqui sobe e fica na casa dos milissegundos ou microssegundos. No MPI, o desempenho não depende apenas do processador mais rápido mas também da eficiência da rede. O speed up sofre com o overhead de comunicação. Alguns comandos do MPI exigem que todos os nós terminem suas tarefas e troquem informações. Se um *worker* possui alta latência, todos os outros nós ficarão ociosos esperando por ele. Um único nó lento atua como uma âncora para o cluster. Em algoritmos iterativos, o tempo de cada iteração será ditado pelo nó de maior latência, impedindo que a aplicação escale linearmente. Identificar a máquina com o menor "ping" em relação ao Mestre é vital porque, em arquiteturas distribuídas, a rede é frequentemente o **gargalo de IO**. Enquanto no multithreading o desafio é evitar condições de corrida (lógica), no MPI o desafio é minimizar o tempo em que as CPUs ficam paradas esperando pacotes chegarem pela rede. Quanto menor a latência, menor o custo de sincronização e maior a eficiência da computação paralela real.

5. Durante o teste de execução paralela do comando echo, a ordem de finalização e impressão no console foi aleatória e muitas vezes intercalada entre os processos de diferentes *ranks*. Analise este fenômeno à luz da teoria de sistemas distribuídos e concorrência: por que a ordem de impressão não é garantida, mesmo que o envio original obedeça ao sequenciamento do for loop do mpirun?

R: Embora o mpirun inicie os processos em um loop sequencial, essa ordem de "disparo" é rapidamente perdida. Assim que os processos são instanciados **cada processo roda em um core ou nó físico independente**. Eles não compartilham o mesmo clock nem o mesmo escalonador de tarefas. Pequenas flutuações na carga da CPU, no acesso à memória ou na latência da interface de rede de cada nó fazem com que um processo iniciado milissegundos depois termine antes de um iniciado primeiro. No

momento em que o comando echo é executado, ele é uma entidade autônoma. O sistema operacional de cada nó decide quando dar tempo de CPU para aquele processo específico, tornando a finalização inerentemente aleatória. **O MPI garante: A ordem de envio de mensagens da mesma origem.** Isso é a regra de *Non-overtaking* (não ultrapassagem) entre um par específico de origem e destino.

❓ **O MPI NÃO garante: A ordem de chegada de mensagens vindas de origens diferentes.** É uma competição livre.

Assunto 3: Comunicação Coletiva em MPI e Processamento Distribuído

Questões Objetivas

1. **[Nível: Análise]** O projeto de detecção de Pneumonia exige a distribuição da matriz da radiografia e dos limiares de suspeita clínica. A estratégia correta adotada utiliza MPI_Bcast para enviar os limiares e MPI_Scatter para distribuir a imagem. Se a estratégia fosse alterada e toda a imagem fosse enviada utilizando APENAS MPI_Bcast, qual seria o impacto arquitetural no sistema?
- a) O tempo de execução seria reduzido, pois o Bcast utiliza uma árvore topológica mais otimizada que o Scatter.
 - b) Ocorreriam falhas de segmentação de memória, pois o Bcast só suporta o tipo MPI_INT.
 - c) Haveria replicação massiva de dados, pois todos os processos receberiam uma cópia completa da imagem, aumentando drasticamente o consumo de memória e a saturação da banda da rede, desperdiçando o princípio do particionamento de dados.
 - d) Não haveria alteração prática, pois Bcast e Scatter agem da mesma forma por baixo dos panos na API mpi4py.

Gabarito: C.

2. **[Nível: Aplicação]** Você é responsável pela Etapa 2 do Laboratório de Logs: a divisão do *Dataset* contendo 100.000 linhas entre 4 processos MPI instanciados. Qual mecanismo coletivo da biblioteca mpi4py deve ser obrigatoriamente chamado pelo coordenador (rank 0) para realizar essa partição de forma igualitária (25.000 linhas cada), onde cada processo termina apenas com sua fração em memória?

- a) `comm.scatter(...)`
- b) `comm.bcast(...)`
- c) `comm.gather(...)`
- d) `comm.sendrecv(...)`

Gabarito: A.

3. [Nível: Aplicação] Na Etapa 10 e 11 do cenário das Radiografias, é necessário coletar dois tipos de informações: 1) o *total* numérico global de pixels suspeitos em todo o cluster; e 2) os *relatórios detalhados individuais* gerados por cada processo (rank, linhas processadas, classificação da faixa). Quais primitivas de comunicação coletiva do MPI devem ser aplicadas, respectivamente, para resolver esses dois pontos com maior elegância?

- a) MPI_Bcast para o total global; MPI_Scatter para os relatórios detalhados individuais.
- b) MPI_Reduce para o total global (consolidando os valores aplicando um operador de soma, por exemplo); MPI_Gather para reunir os relatórios detalhados como estruturas de dados no nó *root*.
- c) MPI_Allgather para ambas as demandas.
- d) MPI_Send e MPI_Recv encadeados em laços aninhados para ambas as demandas.

Gabarito: B.

Questões Discursivas

4. [Nível: Análise]

Enunciado: Para testar a resiliência do algoritmo de análise de radiografias, foi introduzida na Etapa 8 uma simulação de "cluster heterogêneo", onde os nós de rank ímpar receberam um comando sleep (atraso artificial). Na Etapa 9, chamou-se um MPI_Barrier() antes da consolidação dos resultados (Reduce/Gather). Analise o comportamento mecânico que ocorrerá nesta barreira. Se você possui um cluster com 99 nós ultra-rápidos e 1 nó lento, qual será o impacto no cálculo do *Speedup* da sua aplicação e por quê?

Guia de Resposta:

1. Avalie a semântica da função MPI_Barrier(): ela atua como um portão que só se abre quando 100% dos processos (incluindo o mais atrasado) chegam até ela.
2. Aplique essa definição ao cenário do nó "lento". O que farão os 99 nós rápidos assim que concluírem o processamento local? Eles ficarão em estado ocioso esperando o gargalo (nó lento) concluir.
3. Conclua com o princípio de Escalabilidade e *Speedup*: O tempo total da aplicação paralela é sempre ditado pela tarefa de processamento mais longa. Logo, a distribuição desbalanceada afeta massivamente a eficiência paralela.

5. [Nível: Análise]

Enunciado: O algoritmo *Odd-Even Transposition Sort* é uma alternativa paralela baseada no clássico *Bubble Sort*. Em sua implementação com MPI, foi destacado que a utilização de chamadas simples com MPI_Send e MPI_Recv durante as fases de troca de valores entre vizinhos era perigosa e gerava risco de *Deadlock*, exigindo uma substituição por primitivas combinadas (MPI_Sendrecv). Analise

essa falha arquitetural do ponto de vista do "buffering" interno do MPI. Por que a simultaneidade do *send/recv* causa travamento mútuo na ordenação? *Guia de Resposta:*

1. Visualize duas partições de memória adjacentes (Rank 0 e Rank 1) na fase Par. Ambas decidem enviar (Send) dados uma para a outra ao mesmo tempo.
2. Relembre a regra fundamental: se o buffer do sistema estiver cheio ou se a implementação do Send for bloqueante, o Rank 0 aguardará o Rank 1 iniciar o Recv, mas o Rank 1 estará aguardando o Rank 0 iniciar o Recv para poder destravar o seu Send.
3. Deduzir que o Sendrecv resolve isso por agendar o fluxo das vias e proteger atomicamente a troca na rede.